

《膳食搭配智能体系统》

数据库设计说明书

团队名称	海底小纵队
数据库名称	diet_agent（PostgreSQL 18）
文档版本	V1.2
编制日期	2026-11-04
编制人	李铭媛、陈博淇

依从 GB/T 8567—2006

表 1: 修订记录

版本	日期	修改人	说明
V0.1	11/04	李铭媛	建立数据库设计说明书骨架（依 GB/T 8567—2006）
V0.2	11/04	陈博淇	攥写第四节“数据库级设计决策”
V0.3	11/04	李铭媛	攥写其余部分
V0.4	11/05	李铭媛	补充 E-R 图
v0.5	11/05	陈博淇	审查全文，修改细节
v0.6	11/05	李铭媛	整理全文
V1.0	11/08	李铭媛	对齐改进后的原型设计
V1.1	11/11	李铭媛	同步 v1.2 口径（菜库、方案系列表结构）
V1.2	11/14	李铭媛	修正 profile 字段（conditions、goal_code）与病种目录

目录

1 注解 **1**
1.1 术语和定义 1
1.2 缩略语 1

2 引言 **2**
2.1 标识 2
2.2 数据库概述 2
2.3 文档概述 3

3 引用文件 **3**

4 数据库级设计决策 **4**
4.1 数据库管理系统选型 4
4.2 Schema 划分 4
4.3 账号、权限与隐私保护 4
4.4 空值语义 5
4.5 数据质量与过敏原口径 5
4.6 一致性与完整性 6
4.7 备份与恢复 7
4.8 可扩展性 7
4.9 设计约定 7

5 数据库详细设计 **7**
5.1 概念结构设计 7
5.1.1 实体清单 7
5.1.2 实体—联系图（ER） 8
5.2 逻辑结构设计 8
5.2.1 关系表总览 8
5.2.2 范式说明 9
5.2.3 新增表结构与状态机 10
5.2.4 profile 字段 10
5.2.5 健康目标结构映射 11
5.2.6 Schema 划分与权限边界 12
5.2.7 数据元素特性（字段字典） 12
5.3 物理结构设计 12

6 用于数据库访问或操纵的软件配置项的详细设计 **12**

1 注解

本章给出理解本文档所需的术语、定义与缩略语。

1.1 术语和定义

表 2: 术语和定义

术语	定义
规则引擎	不依赖大语言模型的确定性工具链，负责候选搜索、差异化、跨日微量平衡与回溯补位，输出候选结构与营养概要
硬约束	硬约束指过敏、疾病、宗教等饮食禁忌，候选中不得出现
状态机	方案的生命周期模型，依次为 `draft`（生成中）→ `candidates`（候选已生成）→ `fixed`（已固定，本方案只读）
跨日动态平衡	7 种关键微量不再按周达标，而是读取近期历史方案的营养报告，识别不足并在当前生成中补齐，实现跨日平衡
软删除	不物理删除数据行，而以状态字段或删除标记使其退出常规查询，保留既有引用关系
Schema（模式）	PostgreSQL 中的命名空间，用于划分数据归属；本项目分为 `core` / `knowledge` / `app` / `audit` 四个 Schema
主键 / 外键	主键唯一标识一行；外键引用另一表的主键以维护引用完整性
事务	一组要么全部生效、要么全部撤销的数据库操作单元
第三范式（3NF）	非主属性仅依赖主键、不传递依赖的关系设计规范
router（路由模块）	后端按功能划分的接口模块，每个文件承载一组相关接口
SQLAlchemy	Python 的对象关系映射（ORM）框架，后端以此访问数据库

1.2 缩略语

表 3: 缩略语字母序列表

缩略语	含义
3NF	第三范式（Third Normal Form）

表 3: 缩略语字母序列表

缩略语	含义
B-tree	平衡树索引（balanced tree）
DRIs	膳食营养素参考摄入量（Dietary Reference Intakes）
FDC	USDA 食物数据中心（FoodData Central）
GB/T	中华人民共和国国家标准（推荐性）
GIN	通用倒排索引（Generalized Inverted Index）
JSON / jsonb	JavaScript 对象表示法；`jsonb` 为其二进制形式
MVP	最小可行产品（Minimum Viable Product）
ORM	对象关系映射（Object-Relational Mapping）
pg_trgm	PostgreSQL trigram 全文索引扩展
RAG	检索增强生成（Retrieval-Augmented Generation）
SQL	结构化查询语言（Structured Query Language）
SSE	服务器推送事件（Server-Sent Events）
TOAST	PostgreSQL 大字段外置存储机制
UTC	协调世界时（Coordinated Universal Time）

2 引言

2.1 标识

本文档适用的数据库完整标识如下：

- **数据库名称：**`diet_agent`；
- **标识号：**无独立编号；本课程项目以数据库名 `diet_agent` 作为唯一标识；
- **版本号：**v1.2；
- **发行号：**第 2 版。

2.2 数据库概述

1. **用途：**为「膳食搭配智能体系统」提供权威数值基准、文本知识与业务数据的持久化与访问，支撑档案管理、方案生成、营养分析、对话咨询等全部后端读写。

2. **一般性质**：关系型数据库；核心数值与业务关系用规范关系表存储，半结构化内容（食材清单聚合、每餐宏量区间、候选营养估算等）用 `jsonb` 存储；文本知识预留 pgvector 向量检索能力。
3. **在系统中的位置**：数据库位于后端（FastAPI）与权威数据源之间，是系统唯一数据底座。权威数据源（《中国食物成分表》《中国居民膳食指南（2022）》《DRIs 2023》）经数据侧清洗与口径统一后离线入库，运行时不再依赖外部数据源。
4. **数据基线**：128 种食材、60 道菜、16 条做法规则、18 种营养素 / 448 条参考值、26 条知识（知识文档 + 营养素知识）。
5. **开发 / 使用 / 维护历史**：v1.1 首次交付（24 张表 / 5 视图 / 8 函数）；v1.2 依据产品改进方案升级——新增方案、候选等 6 张表 + 1 视图 + 1 函数，改造用户档案表（扁平 11 字段）与方案营养报告表的汇总结构，废弃旧的日 / 周方案表。
6. **相关方**：本系统为课程项目，目标用户为有自炊条件、有意识调控饮食的人群，开发方为「海底小纵队」。
7. **运行现场**：轻量服务器 + Docker 容器化部署，无专用硬件依赖。

2.3 文档概述

1. **用途**：本文档依 GB/T 8567—2006 第 7.14 节《数据库（顶层）设计说明》编制，给出数据库的设计决策、概念 / 逻辑 / 物理三级设计与软件访问配置项设计，作为数据库实现、验收与后续维护的依据。
2. **预期读者**：数据岗（实现与维护）、后端开发（ORM 与访问）、测试与交付（验收）。
3. **保密性**：用户身高、体重、疾病、过敏、禁忌等身体数据属个人敏感信息，须加密存储、最小必要采集，且仅用于生成膳食方案，不向任何第三方提供或共享。

3 引用文件

1. 中华人民共和国国家质量监督检验检疫总局，中国国家标准化管理委员会. GB/T 8567—2006 计算机软件文档编制规范 [S].
2.《膳食搭配智能体系统 • 软件需求规格说明书》（本课程成果，V1.4）。
3.《膳食搭配智能体系统 • 系统设计说明书》（本课程成果）。
4. 中国疾病预防控制中心营养与健康所. 中国食物成分表（标准版）（第 6 版）[M]；补充食材数据参考 USDA FoodData Central（FDC）。
5. 中国营养学会. 中国居民膳食指南（2022）[M]；中国居民膳食营养素参考摄入量（2023 版）[M]。

4 数据库级设计决策

4.1 数据库管理系统选型

项目使用 PostgreSQL 18，数据库名为 `diet_agent`。目标联调环境使用 Docker 容器 `diet-agent-pg`。PostgreSQL 容器内部使用默认端口 5432，主机访问端口根据实际部署环境配置。

PostgreSQL 同时支持关系表、数组和 `jsonb`，能够保存食材、菜式、用户档案和方案等结构化数据，也能保存规则原文、来源快照和计算摘要。主键、外键、检查约束和事务用于保证数据一致，视图和函数用于统一食材搜索、营养计算、过敏筛查和参考值匹配。数据库通过 dump 和 SQL 脚本进行交付与恢复。

4.2 Schema 划分

数据库采用单库、四个 Schema 的组织方式。

表 4: 四个 Schema 的职责与主要数据

Schema	职责	主要数据
`core`	保存规则引擎使用的基础数据和规则参数	食材、菜式、菜式配料、做法规则、营养素、参考值、疾病规则、症状营养映射、健康目标结构映射和数据来源
`knowledge`	保存营养解释和咨询所需的知识资料	营养知识、知识文档和检索字段
`app`	保存用户操作和规则引擎生成的业务数据	用户档案、方案、餐位、候选、候选菜式、食材清单、营养分析结果、方案营养报告、会话和消息
`audit`	保存数据导入和审核记录	导入批次、数据问题和规则审核记录

core 和 knowledge 由数据维护流程更新，应用运行时只读；app 由后端按业务接口读写；audit 用于记录数据来源、导入过程和审核结果。原始 CSV 或 JSON 先完成字段检查、类型转换和单位统一，再写入正式 Schema。

4.3 账号、权限与隐私保护

数据库访问划分为两类账号：后端应用统一使用非超级用户 `diet_app` 连接数据库；疾病 / 营养规则的审核另设独立的审核角色，两者权限分开。

应用账号的 Schema 级权限（`diet_app`）：

- core、knowledge 只开放查询权限；

- app 开放业务所需的查询、新增和更新权限；
- audit 开放必要的记录写入和查询权限。

规则审核角色：疾病 / 营养规则在启用前须经人工审核。审核由区别于 diet_app 的独立数据库角色完成，其调用权限只授予审核后台；审核资格由组织流程核实，数据库仅保存审核记录，不允许绕过审核直接修改规则的启用状态。

Agent 的访问边界：对话 Agent 不直接连接数据库，一律经后端访问。Agent 经后端读取档案、方案与知识，并经确定性工具写入生成、换候选、固定等方案数据与会话消息；所有写方案的操作均由确定性工具执行，Agent 自身不直接写库。

敏感数据与隐私：身体参数、疾病、过敏信息、健康目标与对话内容按敏感数据管理。业务表以 user_id 区分用户，后端在读写时校验数据归属；测试数据与真实用户数据分离；日志不记录完整个人资料。

安全基线：现有安全脚本已关闭 PUBLIC 的默认访问权限；v1.2 新增表建立后补充 diet_app 的对象级授权，并在数据库验收中检查权限。

4.4 空值语义

NULL 表示未知、未提供、未核查或当前不适用，0 表示数据来源确认后的实际零值。

例如，某食材的维生素 C 为 NULL，表示当前数据源没有提供该数值。营养计算和结果展示保留原始空值，同时返回完整性状态或缺失项目。过敏原状态为「未核查」时，按未核查处理。用户尚未填写的档案字段同样保存为 NULL，只有业务规则明确规定时才使用默认值。

4.5 数据质量与过敏原口径

食材表（core.ingredient）以 data_quality 字段记录每条食材数值的数据质量等级（来源可信度），取值分三态，见下表。

表 5: 数据质量三态取值与使用方式

取值	含义	使用方式
official	与官方或项目认可的权威资料直接对应	用于正常计算，并保留具体来源
approx	根据相近食材或既定方法估算	用于量级估计，展示时标明估算属性
check	数据仍需核查	暂不用于确定性的安全判断

过敏原使用「已确认含、确认不含、未核查」三种状态。候选生成时，命中用户过敏原的食材或菜式直接排除；未核查食材不向相关过敏用户标记为安全。

用户疾病信息需要保留疾病名称和确认状态。MVP 暂沿用 jsonb，记录「确诊、疑

似、未知」等状态。后端进行规则判断时只使用符合执行条件的记录。

4.6 一致性与完整性

结构关系与基数：v1.2 的逻辑结构围绕用户、方案、餐位、候选和菜式建立，实体间关系与基数约定如下：

- 一个用户可以拥有多份历史方案；
- 餐位数由生成范围决定，餐位数量与类型由智能体按需编排；生成范围可为一餐、一日、数日或更长；
- 每个餐位保存 2~3 个候选，每个候选由主食和 1~2 道菜组成；候选通过更换主食材或做法形成差异；
- 方案关联食材清单和方案营养报告；方案食材清单根据固定后的方案汇总生成；
- 对话消息可以引用当前或历史方案，引用关系只读。

完整性约束：数据库以外键、唯一约束和检查约束保证表间引用与取值合法。用户约束为硬约束——过敏、疾病、宗教等饮食禁忌在候选中不得出现。MVP 不记录辅料接受程度，辅料按可用处理。

状态机与一致性：方案状态为 `draft` → `candidates` → `fixed`：

- `draft`：规则引擎正在生成；
- `candidates`：候选已经生成，用户可以选择或更换候选；
- `fixed`：方案已经固定，本方案内只读。

固定方案时，系统校验各餐位的候选与选择结果，并在同一事务内写入选定候选、固定时间与最终状态；任一步失败则整体回滚，不产生部分固定的中间状态。食材、菜式与历史方案采用状态字段或软删除（标记删除而非物理删除）管理，以保持既有方案的引用关系；候选解释、营养分析与方案结果保存所依据的数据版本与来源标识，供结论回溯。

业务规则（营养口径）：规则引擎负责候选搜索、差异化、跨日微量平衡和回溯补位，同时输出候选结构、营养概要和微量覆盖进度；对话 Agent 负责调用这些工具、处理生成过程中的冲突，并补充搭配解释和来源；候选调整通过重新调用确定性工具完成，营养数值以工具输出为准。营养口径按 v1.2 方案执行：

- 能量目标按基础代谢率乘固定系数 1.4 估算，用于确定份量量级；
- 宏量营养按餐控制，默认分配为早餐 30%、午餐 40%、晚餐 30%；
- 钙、铁、锌、维生素 A、维生素 B1、维生素 B2 和维生素 C 跨日动态平衡（读取近期历史方案），目标以人群参考值为基础；
- 其他微量营养素用于参考提示；
- 五类健康目标的结构规则存于 core 的映射表 `goal_structure_map`，先录入已明确的区间与倾向。

疾病营养规则经过专业审核并启用后进入规则计算，待审核规则保留在数据库中，供后续核对和完善。

范围界定：MVP 支持生成与换候选（均可由对话触发，经确定性工具执行），方案固定后本方案内只读；不做实际执行追踪、偏离再平衡、外食或跳过标记，相关能力留待后续版本设计。

4.7 备份与恢复

现有数据库中的食材、菜式、营养素、参考值、知识和来源数据通过迁移脚本纳入 v1.2 数据库。v1.2 结构完成后，重新导出结构脚本、基础数据、字段字典、外键关系和完整备份，并进行恢复测试。

建表和基础数据初始化脚本采用可重复执行的方式编写，用于新环境部署和数据库恢复。pgvector 启用后，知识文档的首次向量化纳入初始化流程。

4.8 可扩展性

食材、菜式、健康目标与疾病规则等基础数据以稳定标识为键，并记录版本号、来源与导入批次，使每条数值均可追溯其出处与入库批次；数据更新时保留旧来源与版本记录，供变更历史追溯。

知识库的目标存储方案为 PostgreSQL 与 pgvector。本期先使用关键词检索，确定嵌入模型和向量维度后，再增加向量字段和索引，与关键词结果组成混合检索。

4.9 设计约定

1. **命名**：表名、列名统一小写 + 下划线；主键列 `xx_id`；时间戳 `created_at` / `updated_at` / `fixed_at`；外键列以「被引用表名 _id」命名。
2. **主键类型**：业务流水表用 `bigserial`；核心字典表（dish / ingredient / nutrient）用 `text` 主键，便于与权威数据源 ID 对齐。
3. **时间类型**：日期用 `date`，时间戳用 `timestamptz`（UTC 存储、应用层转本地时区）。
4. **半结构化**：候选宏量区间、食材清单聚合等用 `jsonb`，schema 由应用侧约定。

5 数据库详细设计

本章按「概念结构设计 → 逻辑结构设计 → 物理结构设计」三个级别逐级细化。

5.1 概念结构设计

5.1.1 实体清单

系统核心实体分为三层，共 23 个实体；本清单仅列业务主实体，关联表与导入、审核等辅助表从略，完整表清单见逻辑结构设计的「关系表总览」。

表 6: 核心实体清单

层次	实体
业务层（app）	用户档案 profile、方案 meal_plan、餐位 meal_slot、候选 meal_candidate、候选菜式 meal_candidate_dish、食材清单 shopping_list、营养报告 nutrition_report、会话 conversation、消息 message、营养结果 nutrition_result
权威层（core）	食材 ingredient、菜式 dish、配料 dish_ingredient、做法规则 cooking_rule、营养素 nutrient、营养素参考值 nutrient_reference、健康状况 health_condition、疾病规则 condition_nutrient_rule、症状映射 symptom_nutrient_map、目标结构映射 goal_structure_map
知识层（knowledge）	知识文档 knowledge_doc、营养素知识 nutrient_knowledge

5.1.2 实体—联系图（ER）

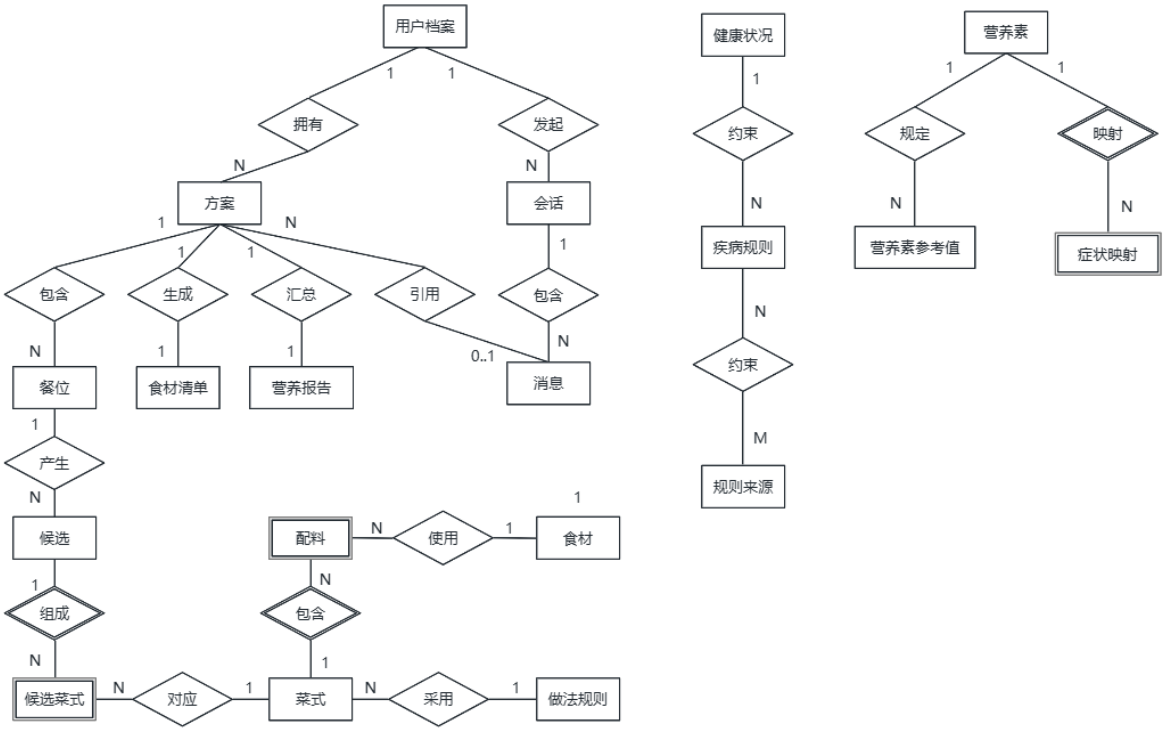


图 1: 核心实体关系图

5.2 逻辑结构设计

5.2.1 关系表总览

表 7: 关系表总览（v1.2，含新增 / 改造标记）

Schema	表 / 视图 / 函数	职责	变更
core	ingredient / dish / dish_ingredient / cooking_rule / nutrient / nutrient_reference / health_condition / condition_nutrient_rule / condition_rule_source / dietary_rule_set / symptom_nutrient_map / data_source / food_id_mapping	权威数值基准	保留（补数据）
core	goal_structure_map	健康目标 → 结构映射	新增
knowledge	knowledge_doc / nutrient_knowledge	文本知识	保留
app	profile	用户档案（扁平 11 字段）	改造
app	meal_plan / meal_slot / meal_candidate / meal_candidate_dish	方案状态机	新增
app	shopping_list	方案食材清单	新增
app	conversation / message / nutrition_result	会话 / 消息 / 营养结果	保留（message 微改）
app	nutrition_report	方案营养报告	改造
app	v_key_micronutrient（视图）	标出 7 种关键微量	新增
app	recent_micro_balance（函数）	跨近期历史方案汇总 7 种关键微量缺口	新增
audit	import_batch / import_issue / condition_rule_review	导入批次 / 导入问题 / 规则审核	保留

5.2.2 范式说明

关系表设计满足第三范式（3NF）：非主属性仅依赖主键、不传递依赖；候选、餐位等通过外键逐级引用，无冗余的跨表字段。半结构化维度（候选宏量区间、食材清单聚合）刻意以 `jsonb` 承载，属反范式取舍：以放弃规范化换取查询与序列化便利，其内部结构由应用层约定。

5.2.3 新增表结构与状态机

表 8: 方案系列新增表结构

表	主键	关键字段	外键
meal_plan	meal_plan_id[1]	user_id、start_date、end_date、status、macro_ratio、fixed_at	—
meal_slot	meal_slot_id[1]	slot_date、meal_type[4]、target_macro、selected_candidate_id	meal_plan_id → meal_plan
meal_candidate	candidate_id[1]	rank[5]、why_text、macro_estimate	meal_slot_id → meal_slot
meal_candidate_dish	(candidate_id, dish_id)[3]	role[6]、servings	candidate_id → meal_candidate; dish_id → core.dish
shopping_list	shopping_list_id[1]	items[7]	meal_plan_id → meal_plan
goal_structure_map	goal[2]	macro_ratio、oil_salt_sugar、food_tendency	—

注：[1] 主键类型 bigserial，自增 8 字节整数；[2] 主键类型 text，字符串；[3] 复合主键以元组 (字段 1, 字段 2) 表示；[4] meal_type 取值不固定（如早/午/晚/加餐等），由智能体编排；[5] rank 取值 1∼3；[6] role 取值主食/菜；[7] items 为 jsonb，按分类分组并含 is_staple 标记；外键列「字段 → 表」表示字段引用该表的主键。

方案状态机：`meal_plan.status` 取值 `draft`（生成中）→ `candidates`（候选可换）→ `fixed`（已固定，本方案锁定只读），由状态字段显式表达，不靠时间推断。

5.2.4 profile 字段

表 9: profile 扁平字段

字段	关键字段	语义
身份	`name / avatar`	昵称、头像
身体参数	`gender / age / height_cm / weight_kg`	算 BMR（Mifflin-St Jeor）

表 9: profile 扁平字段

字段	关键字段	语义
健康目标	goal_code	英文主键 maintain / fat_loss / muscle_gain / glucose_control / digestive_health；中文名（goal_name）维持 / 减脂 / 增肌 / 控糖 / 改善肠胃
烹饪条件	cooking_equipment[]	灶具 / 烤箱 / 微波炉 / 电饭煲 / 空气炸锅
疾病限制（硬）	conditions（jsonb）	对象 {condition_id: {status: confirmed|suspected|unknown}}，key 取 core.health_condition.condition_id；仅 confirmed 参与规则，suspected/unknown 需追问
过敏原（硬）	allergies[]	无 / 花生 / 牛奶 / 鸡蛋 / 海鲜 / 小麦
宗教禁忌（硬）	religious_ethical[]	无 / 清真 / 素食 / 不吃猪肉 / 不吃牛肉
目标热量（派生）	target_calories	BMR × 1.4，系统计算，非用户填写

注：字段名后的 [] 表示 PostgreSQL 数组类型，即该字段一列存一组值（如 allergens[] 存多个过敏原）；不带 [] 的字段为单值；conditions 为 jsonb 对象（非数组），结构见上表。

关键语义：疾病限制、过敏原、宗教禁忌三类均为硬排除（绝不出现）；目标热量由系统按 BMR × 1.4 派生。

5.2.5 健康目标结构映射

表 10: goal_structure_map 映射数值

goal_code	宏量供能比倾向	油盐糖	食材 / 做法倾向
maintain（维持）	碳水 50–65% / 蛋白 10–20% / 脂肪 20–30%	常规	均衡多样
fat_loss（减脂）	蛋白 18–25%、脂肪压至 20% 下限	收紧、控添加糖	低 GI 主食、高纤蔬菜、清淡做法
muscle_gain（增肌）	蛋白 20–25%、优质蛋白占比 ↑	常规	肉蛋奶豆足量、主食保能量

表 10: goal_structure_map 映射数值

goal_code	宏量供能比倾向	油盐糖	食材 / 做法倾向
glucose_control（控糖）	碳水压至 ~50% 下限	收紧、严格控糖	全谷 / 豆 / 薯代精米面
digestive_health（改善肠胃）	纤维 ↑、发酵食物 ↑	油与辛辣 ↓	易消化、少油炸

默认餐次宏量分配：三餐时参考 **早 30% / 午 40% / 晚 30%**；餐位数与宏量按餐由智能体按实际编排灵活调整。能量目标 = BMR × 1.4（活动系数固定），只作「份量量级」参考，不做热量缺口 / 盈余；健康目标只改「结构」。

5.2.6 Schema 划分与权限边界

四个 Schema 位于同一数据库 diet_agent，按数据归属与授权边界分组：core、knowledge 对应用账号只读；app 承载业务表、视图与函数，为应用账号读写区域；audit 承载导入与审核记录，仅追加写入。按 Schema 整体授权，使「权威数据只读、业务数据读写、审计数据只增」的权限规则落在清晰的命名空间边界上。

5.2.7 数据元素特性（字段字典）

全库 376 字段的「名称 / 类型 / 大小 / 单位 / 取值范围 / 精度 / 约束 / 保密性 / 来源」见交付包 05_ 数据库字段字典.csv；新增表的字段字典在 v1.2 补充。

5.3 物理结构设计

1. **存储引擎与存储结构：**PostgreSQL 默认堆存储（heap），jsonb / 长 text 字段由 TOAST 自动外置存储；jsonb 内部二进制 JSON，支持 GIN 索引。
2. **索引：**主键自动建唯一 B-tree 索引；外键列建 B-tree 索引以支撑关联查询；食材 / 菜式搜索列按需建 GIN（jsonb / 数组）与 pg_trgm 全文索引；search_ingredients 等函数内联过滤。

6 用于数据库访问或操纵的软件配置项的详细设计

后端为 FastAPI 应用，接口按功能划分为若干路由模块（router），采用 SQLAlchemy 2.x 与原生 SQL（text()）访问数据库。各 router 访问的库对象映射如下：

表 11: Router 与库表 / 视图 / 函数映射

Router	**访问的表 / 视图 / 函数**
profile.py / users.py	app.profile
ingredients.py	app.search_ingredients、app.v_ingredient_search
dishes.py	app.v_dish_live_nutrition、app.screen_dishes_for_allergens
calc.py	app.v_dish_live_nutrition、core.dish_ingredient、core.ingredient
mealplan.py（新）	app.meal_plan、app.meal_slot、app.meal_candidate、app.meal_candidate_dish、core.dish、core.ingredient、core.goal_structure_map、app.v_key_micronutrient、app.recent_micro_balance
nutrition.py	app.match_nutrient_reference、app.evaluate_weekly_ul、app.nutrition_result
conditions.py	core.health_condition、app.v_condition_rule_catalog、app.v_production_condition_rules、app.evaluate_daily_condition_nutrient_rule
agent.py	knowledge.knowledge_doc、knowledge.nutrient_knowledge
chat.py	app.conversation、app.message

新接口与数据表的读写映射如下：

表 12: 接口与数据表读写映射

接口	读（只读）	写
P-01 / P-02 / P-03 档案	app.profile	app.profile
P-04 食材目录	core.ingredient	—
M-01 生成候选	profile、dish、dish_ingredient、ingredient、cooking_rule、goal_structure_map、nutrient_reference、近期历史方案（meal_plan + nutrition_report）	meal_plan、meal_slot、meal_candidate、meal_candidate_dish
M-02 换候选	meal_plan、meal_slot、meal_candidate	meal_slot.selected_candidate_id

表 12: 接口与数据表读写映射

接口	读（只读）	写
M-03 固定	meal_plan、meal_slot	meal_plan.status / fixed_at、meal_slot.selected_candidate_id
M-04 / M-05 查方案	meal_plan、meal_slot、meal_candidate、meal_candidate_dish、dish	—
M-06 营养分析	meal_plan、nutrition_report、nutrient_reference、v_key_micronutrient、recent_micro_balance	nutrition_report
M-07 食材清单	meal_plan、meal_candidate_dish、dish_ingredient、ingredient	shopping_list
C-01 对话（SSE）	profile（硬约束）、meal_plan（引用）、knowledge	conversation、message
C-02 RAG	knowledge_doc、nutrient_knowledge	—
C-03 反向营养	core.symptom_nutrient_map	—
K-01 营养核算	ingredient、dish、dish_ingredient	nutrition_result

M-01 是唯一「跨 core + app 大量读、写 app」的重接口，其读表清单即生成引擎的数据依赖，数据岗须保证这些表数据完整、字段可查。

附录

A. 支撑素材

1. 空库恢复测试结果与数据库验收脚本：见交付包 `08_ 数据库验收.sql`。
2. 字段字典（376 字段）与外键关系（18 条）：见交付包 `05_ 数据库字段字典.csv`、`06_ 数据库外键关系.csv`，可单独成册。